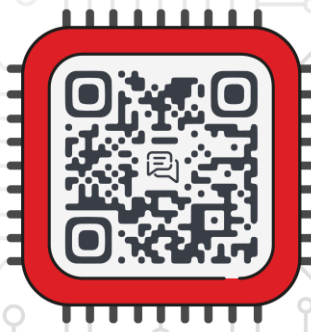




6

25

3000 sp

نظري

كلية الهندسة المعلوماتية

السنة الثانية

Graphics and Java AWT

د. محمد الأحمد



RB Informatics;

4/12/2024

برمجة ٣

السلام عليكم أعزاءنا الطلاب

محاور هذه المحاضرة :

1. Introduction
2. Graphics Contexts and Graphics Objects
3. Color Control
4. Font Control
5. Drawing Lines, Rectangles and Ovals
6. Drawing Arcs
7. Drawing Polygons and Polylines

- سابقاً كنا نتعامل مع التطبيقات من خلال ما يسمى بالـ Command Line Interface (سطر الأوامر / CLI)، وهي واجهة نصية تتيح للمستخدمين إدخال الأوامر النصية مباشرة إلى نظام التشغيل، بدلاً من استخدام واجهة المستخدم الرسومية (GUI).
- في سطر الأوامر، يمكن للمستخدم كتابة أوامر محددة لتنفيذ مهام معينة مثل إدارة الملفات، تشغيل البرامج، أو ضبط إعدادات النظام، وللتفاعل مع هذه الواجهة نحتاج إلى حفظ وكتابة جميع التعليمات والأوامر.
- ومع تطور إصدارات الجافا أصبحنا نتعامل مع الـ GUI وهي اختصار لـ Graphical User Interface (واجهة المستخدم الرسومية) في جافا وهي نوع من واجهات المستخدم التي تسمح للمستخدمين بالتفاعل مع الأجهزة والبرامج بطريقة مرئية بدلاً من استخدام الأوامر النصية فقط، وأصبح التعامل مع هذه الواجهات فعالاً أكثر بحيث نستطيع الضغط على الأزرار وتكبير وتصغير النوافذ وفهم كيفية استخدام التطبيق وغيرها الكثير...

- ال GUI تضم ثلاثة مكتبات (packages) أساسية وهي AWT, swing, javaFx ولكن سنتطرق إلى تعلم هاتين المكتبتين فقط :

1. AWT (Abstract Windowing Toolkit):

- هي أول platform تعاملت معها برامج الجافا بال GUI، وهي عبارة عن واجهة برمجة تطبيقات (API) تضم كلاسات جاهزة (set of predefined classes) تستخدم لبناء وتطوير ال GUIs أو التطبيقات القائمة على النوافذ (window-based application) في جافا.
- تعتمد على المكونات الأولى للنظام الأساسي (Uses peer components of the OS) مما يعني أنها تستخدم عناصر واجهة المستخدم التي يوفرها النظام الذي يعمل عليه التطبيق، وهذا يؤدي إلى جعلها:
 1. Heavy weight: فتأخذ وقتاً طويلاً في التنفيذ.
 2. ليس لها Look and Feel موحد: أي يتغير شكل الواجهة من نظام إلى آخر.
 3. Not truly portable: فقد يظهر التطبيق بشكل مختلف ويتوضع بشكل غير متناسق (lays out inconsistently) على أنظمة تشغيل المختلفة، مما يجعله أقل مرونة.
 4. مكوناتها platform-dependent: فكيفية عرض هذه المكونات تتغير وفقاً لنظام التشغيل الذي يتم تشغيل التطبيق عليه.
- تعمل على جميع أنظمة التشغيل التي تدعم Java.
- نستطيع رسم جميع الأشكال باستخدام هذه المكتبة.
- توفر مجموعة من المكونات مثل: TextField, Label, TextArea, RadioButton, CheckBox, Choice, List..

2. Swing :

- هو إطار عمل (حزمة او مجموعة مكاتب) تم تطويرها كجزء من ال AWT لتحسين Java Foundation Classes (JFC) وتتميز بالعديد من الخصائص مما يجعلها احد الأدوات الأساسية لإنشاء تطبيقات ال JFC.
- تحتوي على مجموعة واسعة من المكونات مثل أشرطة الأدوات (Toolbars) وغيرها.
- باستخدامها يتم رسم المكونات باستخدام Java بدلاً من الاعتماد على مكونات النظام OS، مما يجعلها:
 1. Light weight.
 2. تتميز ب Look and Feel موحد: فتوفر مظهراً موحداً عبر جميع أنظمة التشغيل، حيث تسمح بتغيير نمط الواجهة بسهولة، مما يتيح للمطورين استخدام أنماط مختلفة تناسب تطبيقاتهم.
 3. Platform independent.
- تتميز ال Components في ال Swing بأنها ذات شفافية (Transparency) وجمالية وجاذبية أكثر من ال Components الموجودة في ال AWT ، وأغلب الإمكانيات الموجودة في AWT موجودة في Swing بطريقة أفضل.

- ملاحظة أرسيزية: Java Foundation Classes (JFC) هي مجموعة من أدوات واجهة المستخدم في جافا، توفر إطار عمل متكامل لبناء الواجهات وتتضمن Swing و AWT وغيرها.

تنويه:

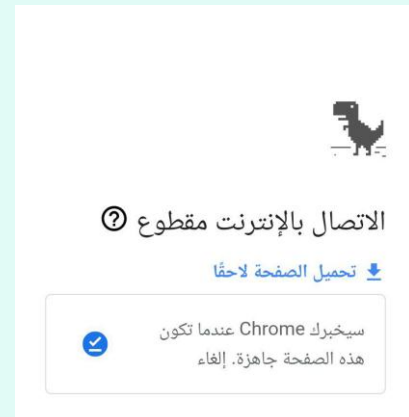
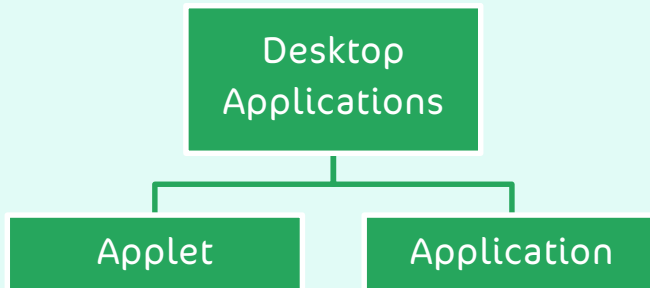
- على الرغم من أن أغلب مكونات AWT تم استبدالها بمكونات من Swing، إلا أنه تم تصميم Swing كامتداد لمكتبة AWT وليس كبديل كامل لها، فتُستخدم بعض الكلاسات في AWT مع الكلاسات الموجودة في Swing لأنها لا تمتلك بدائل في Swing، وسيكون تركيزنا عليها في هذه المحاضرة، مثل:
- (1) رسم الأشكال ثنائية الأبعاد (Drawing 2D shapes)
- (2) التحكم في الألوان (Controlling colors)
- (3) التحكم في الخطوط (Controlling fonts)
- لذا، من الأفضل استدعاء الحزميتين معاً، لأنه قد نحتاج إلى استخدام مكونات Swing وفي نفس الوقت التحكم بالألوان، وهو أمر غير متاح إلا في حزمة AWT ويتم استدعاء الحزميتين كما يلي:

```
import java.awt.*;
import javax.swing.*;
```

- لا يفضل استخدام (مزج) مكونات الحزمة AWT ومكونات الحزمة Swing في نفس التطبيق، حيث إن ذلك قد يتسبب في حدوث مشاكل في التوافق، مثل أخطاء في التخطيط أو العرض، بسبب الاختلافات في طريقة عمل كل من AWT و Swing
- على سبيل المثال: عند استخدام Label في AWT يتم رسمها بواسطة النظام الأساسي وعند استخدام JTextField في Swing يتم رسمها بالكامل بواسطة Java فهذا سيؤدي إلى مشاكل، ولتجنبها يفضل استخدام مكونات إحدى الحزميتين فقط .

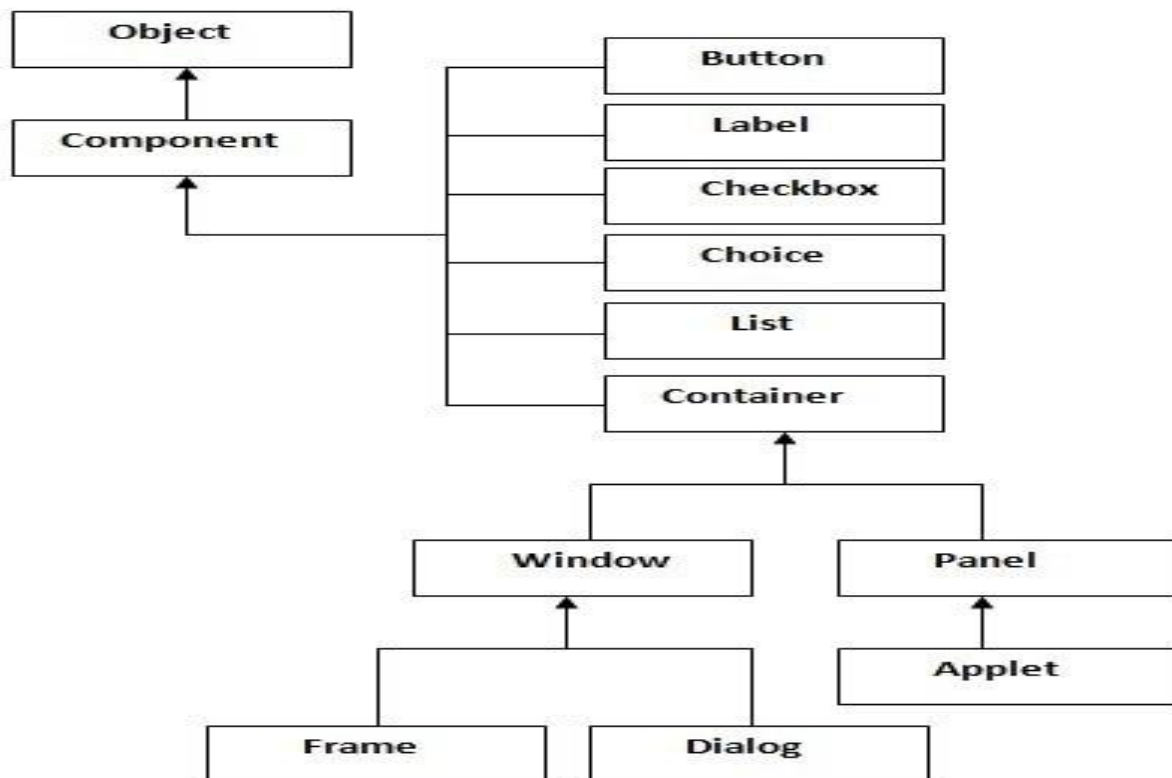
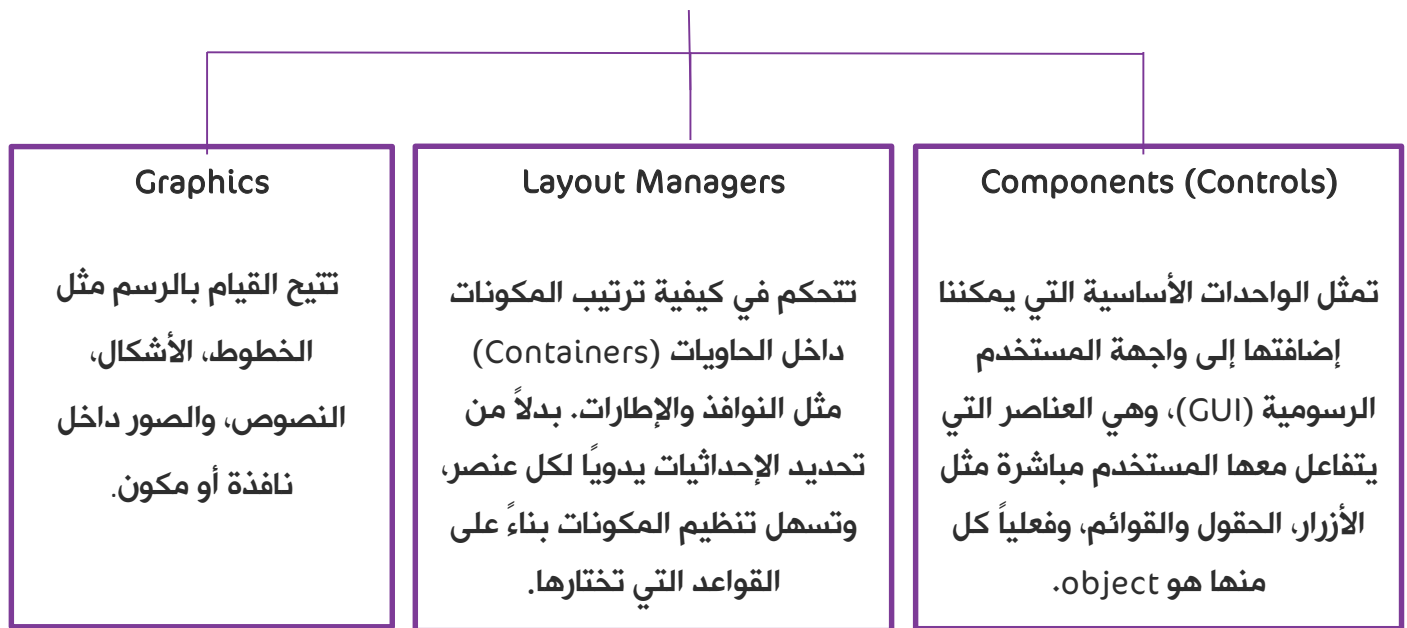
ملاحظات :

- Javafx** : هو إطار عمل حديث لبرمجة الواجهات الرسومية، تم تصميمه ليكون بديلاً عن الـ swing K يوفر ميزات متقدمة مثل الرسوم المتحركة، ولكن نكتفي بدراسة الـ AWT والـ Swing التي تُستخدم بشكل أساسي للتطبيقات المكتبية والـ Applet.
- Applet (تصغير application)**: هو نوع خاص من تطبيقات الـ Java التي تستخدم عادة لتشغيل التطبيقات الصغيرة داخل متصفح الويب، أو عن طريق tool تدعى (Java viewer/Applet viewer) ويمكن تشغيله مباشرة دون الحاجة إلى تثبيت برنامج إضافي. من أمثله لعبة الديناصور التي تظهر في المتصفح عند انقطاع الإنترنت.



■ نلاحظ أن ال swing مستقلة بحد ذاتها لأنها كُتبت بالكامل بلغة الجافا أما ال AWT كانت قد كُتبت بلغة ال C (تعتمد على مكونات مكتوبة بلغة C) فهي غير مستقلة. نميز انه عند استخدام إحدى مكونات ال AWT ننشئ أوبجكت من اسم الكلاس، فعلى سبيل المثال لننشئ كائناً من المكون Label : `Label l=new Label();` اما في ال swing نجد ان الاختلاف فقط بإضافة ال (J) لاسم الكلاس `JLabel l=new JLabel();`

تم تقسيم AWT إلى ثلاث فئات رئيسية (3 categories) لعكس الوظائف الأساسية التي توفرها هذه الحزمة:



Container:

حاويات ترث مباشرة من الكلاس المجرد Component ، وهي نوافذ رئيسية أو فرعية يمكنها احتواء مكونات أخرى (Components) داخلها، إذ لا يمكن إظهار أي شكل أو مكون إلا في container.

أنواع الحاويات (Containers) :

■ Panel:

- حاوية فرعية بسيطة لتنظيم المكونات داخل حاوية أكبر، ولا يحتوي الكلاس Panel على خاصية العنوان (Title) ولا شريط القوائم الذي يعرض قائمة من الخيارات (menubar) ولا الحدود (border)، كما أنه concrete class (غير مجرد فيتم إنشاء الكائنات مباشرة منه).
- يمكن للـ Panel أن تحوي عدة panels أخرى، مما ينشئ مناطق إظهار متعددة ويساهم في التحكم بالمحتوى.
- يرث Applet من Panel، وهو جزء من مكتبة AWT ، ولكن لم يعد مستخدماً كثيراً كالسابق.
- وكما ذكرنا أن Applet هو نوع خاص من التطبيقات الصغيرة (البرمجيات) التي تعمل داخل متصفح ويب، فتتحوّل ضمناً إلى صفحة HTML .
- HTML: ليست لغة برمجية وإنما هي لغة ترميز وإظهار، من مكوناتها الأساسية الـ Tags ولا يتم استخدام الحلقات فيها ولا أسلوب الشرط.

ملاحظة :

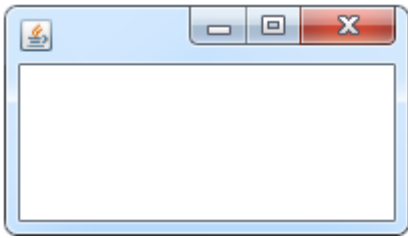
- لا يحتوي الكلاس Applet على الميثود main، فعند تشغيل Applet يتم استدعاء مجموعة من الطرق (methods) المحددة بدلاً من main وهي :
 1. init () : تُستدعى عند بدء تشغيل الـ applet، حيث يمكنك إعداد المكونات الأولية.
 2. Start() : تستدعى بعد init() وتستخدم لبدء أي عمليات تحتاج إلى أن تعمل عندما يكون الـ applet مرئياً.
 3. Stop() : تستدعى عندما يتم إخفاء الـ applet أو عندما يغادر المستخدم الصفحة.
 4. destroy() : تستدعى عندما يتم إزالة الـ applet من الذاكرة.
- لسنا مطالبين بحفظ الميثودات التي تستخدم الـ Applet ولكنها للاطلاع.

■ Window:

حاوية لا تحتوي على حاوية أب لها (Top-level container) ، وتُستخدم لإنشاء نافذة مخصصة بدون أي إطار أي لا تحتوي على شريط عنوان أو أزرار التحكم (Minimize, Maximize, Close) ، وهو Abstract class فلتتعامل معه نعتد على الكلاسات التي ترث منه، ومنها:

1. Frame:

- هو نافذة مستقلة (إطار) يمكنها أن تحتوي على عناصر واجهة المستخدم (مثل الأزرار، النصوص، وغيرها).
- يستخدم لإنشاء التطبيقات الرئيسية التي تحتوي على واجهة رسومية.
- يمتلك شريط عنوان (Title Bar) يمكن تخصيصه، بالإضافة إلى حدود النافذة التي يمكن تصغيرها أو تكبيرها
- تحتوي على أزرار مثل الإغلاق (Close) والتصغير (Minimize) والتكبير (Maximize).
- يمكن بعد إنشائها التحكم في حجمها أو في إظهارها من عدمه.
- لا يمكن وضع Frame خارجية وبداخلها Frame أخرى .
- من الممكن ألا يحتوي الإطار (frame) على أي (panel)، ولكن يُفضل وجود Panel واحدة على الأقل، حيث تساعد في تنظيم المكونات داخل الإطار بشكل أفضل.
- تعريف الـ Frame : `Frame frame = new Frame();`



ملاحظة : هناك عدة اشكال (Overloads) لـ (Constructors) الـ Frame ، منها:

- `Frame ()`
- `Frame(String title)` لتحديد عنوان الإطار.
- `Frame(GraphicsConfiguration gc)` لتحديد إعدادات الرسومات مثل الألوان والأبعاد.
- `Frame(String title, GraphicsConfiguration gc)` لتحديد إعدادات الرسومات مثل الألوان والأبعاد والعنوان.

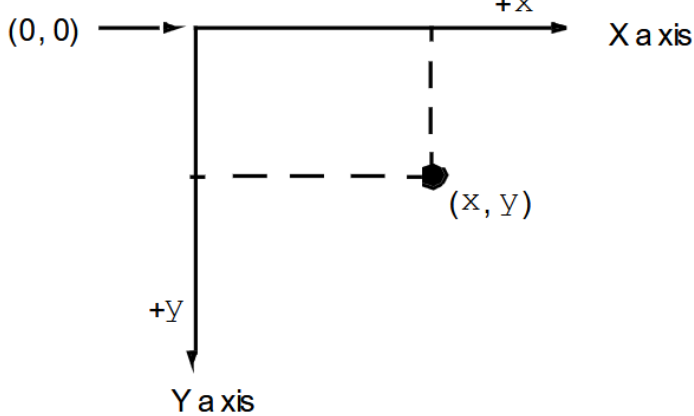
2. Dialog:

- هو نافذة فرعية تابعة لنافذة أولية، تُستخدم عادةً لعرض الرسائل أو التأكيد أو جمع المدخلات من المستخدم، رغم أن الإدخال عن طريقها غالباً غير فعال.
- لا يمكن أن تكون Dialog مستقلة. يجب أن تكون مرتبطة بنافذة أصل (مثل Frame).
- بعض مربعات الحوار تُغلق تلقائياً بعد أن يقوم المستخدم بإجراء معين، مثل الضغط على زر "OK" أو "Cancel" وعدم إغلاق مربعات الحوار يمنع المستخدم من التفاعل مع واجهة المستخدم الرئيسية لذلك يجب إغلاقها .

النظام الإحداثي في جافا Java's coordinate system

هو النظام الذي تستخدمه Java لتحديد مواقع المكونات مثل الأزرار أو الصور أو العناصر الرسومية مثل الأشكال والنصوص على الشاشة أو في نافذة رسومية (scheme for identifying all points on screen).

Coordinate point composed of x-coordinate and y coordinate ، أي أن :



Java تعتمد نظام إحداثيات ثنائي الأبعاد، وكل نقطة في النظام يمثلها زوج من القيم (X, Y) ووحدة القياس هي البيكسل (pixel)، حيث :

- X: يزداد بالتحرك من اليسار إلى اليمين.
- Y: يزداد بالتحرك من الأعلى إلى الأسفل.
- النقطة $(0,0)$ تقع في الزاوية العلوية اليسرى للنافذة (Upper-left corner of frame, applet, component has coordinates $(0,0)$)

يجب الانتباه عند تعيين موضع النقاط في النظام الإحداثي في جافا، حيث أن النظام يظهر فقط الربع الأول من الإحداثيات. على سبيل المثال، إذا قمت بتعيين النقطة $(-1, -1)$ ، فلن تظهر على الشاشة لأنها تقع خارج حدود النظام الإحداثي، كما يجب أن تكون القيم المستخدمة من نوع `int`.

البيكسل (Pixel): هو الوحدة الأساسية لقياس الصور الرقمية. يُعتبر البيكسل نقطة صغيرة جداً على الشاشة، وكلما زادت عدد البيكسلات في الصورة زادت دقتها ووضوحها.

- فائدة: في بايثون، يعتمد نظام الإحداثيات على المكتبة المستخدمة؛ فعلى سبيل المثال، في مكتبة `matplotlib` و `turtle`، يقع المركز $(0,0)$ في منتصف الرسم البياني، بينما في مكتبة `pygame` يكون الزاوية العليا اليسرى هي $(0,0)$.

Graphics Contexts and Graphics Objects

Graphics Context

هو السياق الذي يتم رسم الرسومات فيه (ضمن المنطقة المخصصة للرسم)، ويوفر جميع الأدوات والخصائص اللازمة لرسم العناصر في الواجهة (Enables drawing on screen).

Graphics Object

كائن يمثل الرسومات في جافا، يحتفظ بالمعلومات الضرورية للرسم كإحداثيات اللون والخط ليدير سياق الرسم (manages graphics context).

- إن الكلاس Graphics هو abstract class، لذا لا يمكن إنشاء كائن منه بشكل مباشر (Cannot be instantiated)، بل نستخدمه عن طريق أخذ reference منه واستخدامه مع الميثود paint، وهي ميثود في كلاس Component.

public void paint (Graphics g)

- يساهم كلاس Graphics في جعل جافا أفضل من حيث قابلية النقل (portability)، أي إمكانية عملها على أنظمة التشغيل المختلفة.

سنحدث عن الكلاسات غير المتاحة في Swing والتي نحتاج إلى استدعائها من AWT، وهي :

1. Color Control:

في Java يمكن التحكم بالألوان (manipulating colors) من خلال استخدام الكلاس Color الموجود في الحزمة java.awt

الخصائص الأساسية للكلاس color :

1. إنشاء ألوان محددة مسبقاً: الكلاس Color يوفر ألواناً ثابتة (constants) يمكن استخدامها مباشرة وهي عبارة عن 13 لون ثابت كما في الجدول الآتي :

Color constant	Color	RGB value
public final static Color ORANGE	orange	255, 200, 0
public final static Color PINK	pink	255, 175, 175
public final static Color CYAN	cyan	0, 255, 255
public final static Color MAGENTA	magenta	255, 0, 255
public final static Color YELLOW	yellow	255, 255, 0
public final static Color BLACK	black	0, 0, 0
public final static Color WHITE	white	255, 255, 255
public final static Color GRAY	gray	128, 128, 128
public final static Color LIGHT_GRAY	light gray	192, 192, 192
public final static Color DARK_GRAY	dark gray	64, 64, 64
public final static Color RED	red	255, 0, 0
public final static Color GREEN	green	0, 255, 0
public final static Color BLUE	blue	0, 0, 255

طريقة الاستخدام (اللون.اسم الكلاس) مثل : Color.YELLOW

2. إنشاء ألوان مخصصة باستخدام قيم RGB: يمكن تحديد اللون باستخدام قيم الأحمر (Red)، والأخضر (Green)، والأزرق (Blue)، وهي قيم تتراوح بين 0 و 255، حيث يكون كل لون خليطاً من هذه الألوان الثلاثة. Colors are also created from Red, Green and Blue components, RGB values (0-255)

مثل : Color customColor = new Color(255, 100, 150);

أي أن نسبة مساهمة اللون الأحمر هي 255 واللون الأزرق هي 100 واللون الأخضر 150

■ للحصول على اللون الأسود :

```
Color customColor = new Color(0, 0,0);
```

■ للحصول على اللون الأبيض :

```
Color customColor = new Color(255, 255,255);
```

■ للحصول على اللون الأحمر :

```
Color customColor = new Color(255, 0,0);
```

■ وأيضاً من أجل الحصول على اللون الأخضر نضع جميع القيم 0 ما عدا القيمة الأخيرة التي تمثل اللون الأخضر تكون بنسبة 255، وللحصول على الأزرق نضع جميع القيم 0 ما عدا القيمة الثانية التي تمثل اللون الأزرق تكون بنسبة 255.

■ أما باقي الألوان فتكون النسب بقيم مختلفة وهكذا، ومن المهم معرفة القيم الحدية للألوان الأساسية.

3. إنشاء ألوان مخصصة باستخدام قيم float: اي استخدام القيم كأرقام عشرية بين 0.0 و 1.0

مثل :

```
Color customColor = new Color(1.0f, 0.5f, 0.6f);
```


اللون الأسود :

```
Color customColor = new Color(0.0f, 0. 0f, 0.0f);
```

اللون الأبيض :

```
Color customColor = new Color(1.0f, 1. 0f, 1.0f)
```

■ فائدة أربيسيزية :

إلى جانب قيم RGB ، يمكن إضافة شفافية باستخدام القيمة Alpha .
البارامتر الرابع (alpha) يحدد مستوى شفافية اللون باستخدام قيمة تتراوح بين 0 و 255
القيمة 0 : اللون شفاف بالكامل (غير مرئي)
القيمة 255 : اللون معتم بالكامل (غير شفاف)
مثل :

```
Color customColor = new Color(255, 100, 150, 128);
```

Color methods and color-related Graphics methods:

Method	Description
Color constructors and methods:	
<code>public Color(int r, int g, int b)</code>	إنشاء كائن لون (Color) باستخدام قيم صحيحة (integer values) للمكونات الثلاثة (Red, Green, Blue). القيم يجب أن تكون بين 0 و 255. Creates a color based on red, green and blue components expressed as integers from 0 to 255.
<code>public Color (float r, float g, float b)</code>	إنشاء كائن لون (Color) باستخدام قيم عائمة (float values) للمكونات الثلاثة (Red, Green, Blue). القيم يجب أن تكون بين 0.0 و 1.0. Creates a color based on red, green and blue components expressed as floating-point values from 0.0 to 1.0
<code>public int getRed()</code>	تُستخدم لاستخراج قيم اللون الأحمر من الكائن Returns a value between 0 and 255 representing the red content
<code>public int getGreen()</code>	تُستخدم لاستخراج قيم اللون الأخضر من الكائن Returns a value between 0 and 255 representing the green content
<code>public int getBlue()</code>	تُستخدم لاستخراج قيم اللون الأزرق من الكائن Returns a value between 0 and 255 representing the blue content
Graphics methods for manipulating colors:	
<code>public Color getColor()</code>	تُستخدم للحصول على اللون الحالي المُستخدم في الرسم، فتعيد كائناً من نوع Color. Returns a Color object representing the current color for the graphics context.
<code>public void setColor(Color c)</code>	تُستخدم لتعيين لون جديد للرسومات Sets the current color for drawing with the graphics context.

■ ملاحظة : اللون الأخير المستخدم في الكتابة والرسم هو الذي يبقى مستخدماً حتى يتم استدعاء الدالة `setColor` لتغيير اللون.

■ مثال عن استخدام الألوان في Java :

```

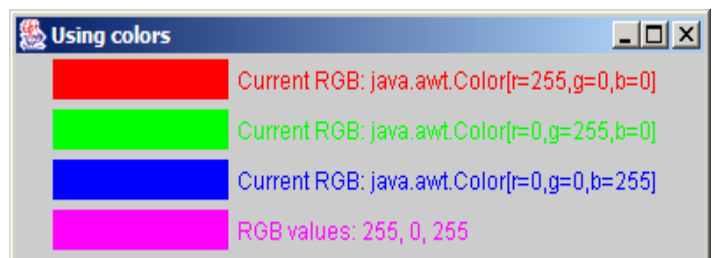
1 // AWT (Abstract Window Toolkit) مكتبة
  التي تحتوي على أدوات الرسومات مثل Color و Graphics
2 import java.awt.*;
3 // استيراد المكتبة Swing لإنشاء النوافذ والمكونات
4 import javax.swing.*;
5 // إنشاء كلاس يرث من JFrame لإنشاء نافذة رسومية
  (إطار)
6 public class test extends JFrame {
7     public test() {
8         // تعيين عنوان النافذة إلى "Using colors" من خلال
  JFrame كونسركتور الأب
9         super("Using colors");
10        // تحديد أبعاد النافذة 400 عرضاً و130 ارتفاعاً
11        setSize(400, 130);
12        // جعل النافذة مرئية
13        setVisible(true);
14    }
15
16    // الدالة paint تُستخدم لرسم النصوص والخطوط
17    public void paint (Graphics g)
18    {
19
20        super.paint(g);
21        // تغيير اللون الحالي للرسم إلى الأحمر باستخدام قيم
  RGB (255, 0, 0)
22        g.setColor (new Color(255, 0, 0));
23        // رسم مستطيل أحمر اللون عند النقطة (25, 25)
  بأبعاد 20×100
24        g.fillRect(25, 25, 100, 20);
25        // طباعة نص يعرض اللون الحالي Current RGB مع قيمه
26        g.drawString ("Current RGB: " + g.getColor(),
  130, 40);
27        // تغيير اللون إلى الأخضر باستخدام قيم نسبية
  (float) 1.0 و 0.0
28        g.setColor(new Color(0.0f, 1.0f, 0.0f));
29        // رسم مستطيل أخضر عند النقطة (50, 25)
  بأبعاد 20×100
30        g.fillRect(25, 50, 100, 20);

```

```

31        // عرض النص مع قيم اللون الجديد
32        g.drawString("Current RGB: " +
  g.getColor(), 130, 65);
33        // تغيير اللون إلى الأزرق باستخدام الكائن الثابت
  Color.BLUE
34        g.setColor (Color.BLUE);
35        // رسم مستطيلاً أزرق عند النقطة (25, 75)
36        g.fillRect(25, 75, 100, 20);
37        // عرض النص مع قيم اللون الأزرق
38        g.drawString("Current RGB: " +
  g.getColor(), 130, 90);
39
40        // تعريف كائن Color باللون MAGENTA
41        Color color = Color.MAGENTA;
42        // تغيير اللون الحالي إلى MAGENTA
43        g.setColor(color);
44        // رسم مستطيلاً بهذا اللون عند النقطة (25, 100)
45        g.fillRect(25, 100, 100, 20);
46        // عرض قيم اللون الفردية (Red, Green, Blue)
  باستخدام التتابع getRed(), getGreen(), و getBlue().
47        g.drawString("RGB values: " +
  color.getRed() + ", " + color.getGreen() + ", "
  + color.getBlue(), 130, 115);
48    }
49
50    public static void main(String args[]) {
51        // إنشاء كائن من الصف test، مما يؤدي إلى عرض النافذة
52        test application = new test();
53        // تحديد أن التطبيق يجب أن يُغلق عند النقر على زر
  الإغلاق (X) الخاص بالنافذة
54        application.setDefaultCloseOperation
  (JFrame.EXIT_ON_CLOSE);
55    }
56 }

```



ملاحظة :

- `drawstring`: دالة تستخدم لرسم نص على واجهة المستخدم

```
g.drawString("hi", 4, 3);
```

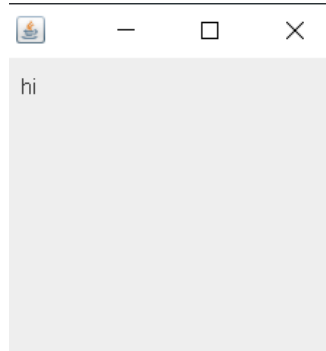
حيث:

"hi" هو النص الذي سيتم رسمه

$4=X$ مما يعني أن النص سيبدأ عند النقطة 4 على المحور الأفقي

$3=Y$ يعني أن النص سيبدأ عند النقطة 3 على المحور العمودي

وبما أن إحداثيات X و Y تبدأ من الزاوية العلوية اليسرى للنافذة. لذا، القيم الصغيرة مثل (4, 3) ستضع النص قريباً من الزاوية كما في الشكل :



- `JColorChooser`: أحد عناصر واجهة المستخدم في Swing، حيث يسمح للمستخدم اختيار لون من بين مجموعة من الألوان.

- `showDialog`: دالة static في كلاس `JColorChooser`، وتستخدم لعرض مربع حوار (dialog) للاختيار اللون.



ارفع راسك فوق.. انت سوري حر ☺

المثال 2: استخدام مربع حوار يسمح للمستخدم باختيار الألوان:

```

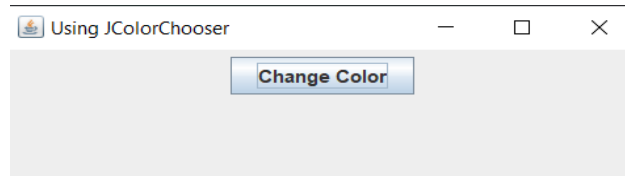
1  import java.awt.*;
2  //ActionEvent استيراد أحداث الواجهة الرسومية مثل
   ActionListener
3  import java.awt.event.*;
4  import javax.swing.*;
5  public class Main extends JFrame {
6  //changeColorButton (Button) يسمى تعريف زر
7      private JButton changeColorButton;
8      // تعريف كائن Color مع تعيين اللون الافتراضي إلى
       LIGHT_GRAY
9      private Color color = Color.LIGHT_GRAY;
10     // تعريف كائن Container لتحديد محتوى النافذة
11     private Container container;
12     public Main() {
13 // "Using عنوان النافذة ليكون
       JColorChooser"
14         super("Using JColorChooser");
15         // الحصول على محتوى النافذة باستخدام
       getContentPane()
16         container = getContentPane();
17         //FlowLayout تعيين تخطيط النافذة ليكون
       لترتيب المكونات بشكل أفقي
18         container.setLayout(new FlowLayout());
19         // "Change Color" إنشاء زر جديد بعنوان
20         changeColorButton = new JButton
       ("Change Color");
21         //addActionListener باستخدام
22         changeColorButton.addActionListener(
23         // تعريف "صف داخلي مجهول Anonymous
       لتحديد منطق معالج الأحداث
24         new ActionListener(){
25             // عند الضغط على الزر، يتم استدعاء الدالة
       actionPerformed
26         public void actionPerformed (ActionEvent
       event) {

```

```

27         // يظهر مربع الحوار JColorChooser حيث:
28         Main.this يشير إلى الإطار الحالي كمالك لمربع الحوار
29         "Choose a color" هو العنوان الذي يظهر في
       مربع الحوار
30         color = JColorChooser.showDialog(
31         Main.this, "Choose a color", color);
32         // إذا لم يختار المستخدم لوناً (النقر على إلغاء)، يتم
       تعيين اللون إلى LIGHT_GRAY
33         if (color == null)
34             color = Color.LIGHT_GRAY;
35         // غير لون خلفية محتوى النافذة إلى اللون الذي
       اختاره المستخدم
36         container.setBackground(color);
37         }}
38     );
39     // إضافة الزر changeColorButton إلى النافذة.
40     container.add(changeColorButton);
41     // تحديد حجم النافذة بـ 400 عرضاً و130 ارتفاعاً
42     setSize(400, 130);
43     // جعل النافذة مرئية
44     setVisible(true);
45 }
46 public static void main(String args[])
47 {
48     Main application = new Main();
49     // تحديد أن التطبيق يغلق عند النقر على زر الإغلاق
       للنافذة (X)
50     application.setDefaultCloseOperation
       (JFrame.EXIT_ON_CLOSE);
51 }

```



2. Font Control:

Font هو كلاس وجزء من الحزمة java.awt، يُستخدم لإنشاء كائنات خطوط وإدارتها
كل كائن Font يحتوي على معلومات، مثل:

1. اسم الخط (Font Name) :
مثل: Serif-SansSerif-Monospaced

شرح آربسيزي:

- Serif: تحتوي زوائد أو خطوط زخرفية، مثل "Times New Roman"
- SansSerif: لا تحتوي زوائد أو أجزاء زخرفية، مثل: "Arial"
- Monospaced: كل حرف فيها يشغل نفس المسافة الأفقية، مثل: "Courier New"

2. أسلوب الخط (Font Style) :
مثل: عادي (plain)، عريض (Bold)، أو مائل (Italic)

3. حجم الخط (Font Size) :
يقاس بالنقاط (points) حيث كل نقطة (1/72 of inch).
مثل: 12 نقطة (12 pt)، 16 نقطة (16 pt)، وهكذا..

Constructores الخاص بالكلاس Font :

Font constructor takes three arguments :

- **Font name:** Monospaced, SansSerif, Serif, etc.
- **Font style:** Font.PLAIN, Font.ITALIC and Font.BOLD.
- **Font size:** Measured in points (1/72 of inch).

Font (String name, int style, int size)

ملاحظة :

- تضمن جافا توفر بعض الخطوط الأساسية، مثل: Monospaced, SansSerif, Serif, Dialog, DialogInput.
- الخطوط المتاحة في كلاس Font تعتمد على النظام الذي يعمل عليه البرنامج (مثل Windows ، Linux ، macOS) أي ليست جميع الخطوط المتاحة في برنامج ال word هي متاحة في الكلاس Font وإذا قمنا بكتابة اسم خط غير متاح (Specifying unavailable font) على النظام (أي في كلاس Font) كـ Times new Roman ، فإن Java ستقوم بتحويله إلى الخط الافتراضي وغالباً هو Serif .

- واحدة قياس الخط في الكلاس Font هي النقطة (point) ، فعلى سبيل المثال :

```
Font font = new Font("Arial", Font.PLAIN, 20);
```

أي إنشاء كائن Font بحجم 20 نقطة.

Font-related methods and constants

- تنويه: بالإضافة للجدول التالي يُفضل قراءة الجدول باللغة الإنكليزية من محاضرة الدكتور (الاسلايد 18).

Method or constant	Description
public final static int PLAIN	يستخدم لتعيين الخط ليكون عادياً
public final static int BOLD	يستخدم لتعيين النص ليكون عريضاً
public final static int ITALIC	يستخدم لتعيين النص ليكون مائلاً
public Font (String name, int style, int size)	التابع البناء للكلاس Font
public int getStyle()	يعيد أسلوب الخط الحالي.
public int getSize()	يعيد حجم الخط الحالي.
public String getName()	يعيد اسم نوع الخط الحالي.
public String getFamily()	تُعِيد اسم عائلة الخط المستخدم في كائن Font
public boolean isPlain()	يعيد true إذا كان الخط عادياً
public boolean isBold()	يعيد true إذا كان الخط عريضاً
public boolean isItalic()	يعيد true إذا كان الخط مائلاً.

ملاحظة آرسيزية:

- أول 3 حقول في الجدول هي constants، أي ثوابت لا يمكن تغيير قيمتها، وسنرى كيفية استخدامها في المثال التالي.
- Family Fonts: هي مجموعة من الخطوط التي تشترك في تصميم أساسي واحد، ولكن تختلف في بعض الخصائص مثل:

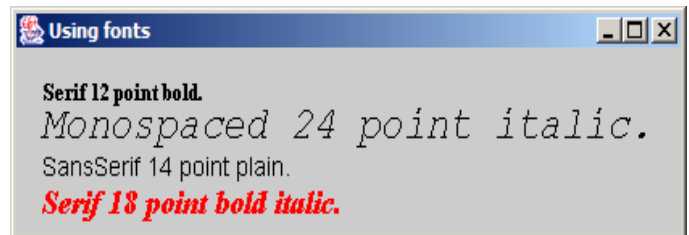
1. سماكة الخط: حيث كلما ازدادت هذه السماكة توحى بأن الكلمات بارزة أكثر.
2. عرض الخط: لأنه هناك خطوط عريضة وأخرى ضيقة وهذا يؤثر على كيفية توزيع النص في المساحة المتاحة.
3. زاوية الخط: أي الميل أو الاتجاه الذي يتم فيه رسم الحرف.

■ مثال عن استخدام الخطوط (Fonts) في Java :

```

1  // البرنامج يهدف إلى استخدام الخطوط
2  import java.awt.*;
3  import javax.swing.*;
4  public class Main extends JFrame {
5      public Main() { // عنوان النافذة
6          super("Using fonts");
7          // حدد أبعاد النافذة بـ 420 عرضاً و125 ارتفاعاً
8          setSize(420, 125);
9          setVisible(true);
10     }
11     public void paint(Graphics g) {
12         super.paint(g);
13         // ضبط الخط إلى Serif ، بحجم 12 نقطة
14         // وبأسلوب عريض (Bold)
15         g.setFont (new Font ("Serif",
16                               Font.BOLD, 12));
17         // رسم النص "Serif 12 point bold."
18         // عند الموقع (20, 50)
19         g.drawString ("Serif 12 point bold.",
20                      20, 50);
21         // ضبط الخط إلى Monospaced بحجم 24
22         // نقطة وبأسلوب مائل (Italic)
23         g.setFont(new Font("Monospaced",
24                             Font.ITALIC, 24));
25         // رسم النص "Monospaced 24
26         // point italic." عند الموقع (20, 70)
27         g.drawString("Monospaced 24
28                      point italic.", 20, 70);
29         // ضبط الخط إلى SansSerif ، بحجم 14
30         // نقطة وبأسلوب عادي (Plain)
31         g.setFont(new Font("SansSerif",
32                             Font.PLAIN, 14));
33         // رسم النص "SansSerif 14 point
34         // plain." عند الموقع (20, 90)
35         g.drawString("SansSerif 14 point
36                      plain.", 20, 90);
37         // تغيير لون النص إلى الأحمر باستخدام
38         g.setColor(Color.RED);
39         // ضبط الخط إلى Serif ، بحجم 18 نقطة،
40         // وبأسلوب عريض ومائل معاً (Font.BOLD +
41         Font.ITALIC)
42         g.setFont(new Font("Serif",
43                             Font.BOLD + Font.ITALIC, 18));
44         // رسم نص ديناميكي يعرض اسم الخط،
45         // وحجمه عند الموقع (20, 110)
46         g.drawString(g.getFont().getName()
47                      + " " + g.getFont().getSize() +
48                      " point bold italic.", 20, 110);
49     }
50     public static void main(String args[]) {
51         Main application = new Main();
52         application.setDefaultCloseOperation
53             (JFrame.EXIT_ON_CLOSE);
54     }
55 }

```



3. Class Graphics :

يستخدم للرسم ومعالجة الرسومات في جافا ، وهو Abstract class لذلك لا يمكن إنشاء كائن من كلاس Graphics مباشرةً فنقوم بإنشاء Reference منه، لأن هذا الكلاس يستخدم بشكل أساسي كوسيلة للرسم داخل دوال معينة مثل paint ، فعند استدعاء هذه الدالة يتم تمرير Reference من نوع Graphics تلقائياً. كل مكون (Component) في AWT يحتوي على كائن Graphics، وهذا ما يعرف بالـ Graphics Context الذي تحدثنا عنه سابقاً، ويمكننا الحصول على هذا الكائن عن طريق استدعاء الدالة getGraphics().

Class Graphics تحتوي على عدة methods للرسم مثل:

- drawLine : لرسم خط
- fillRect ، drawRect : لرسم مستطيلات مفرغة أو مملوءة
- fillOval ، drawOval : لرسم الدوائر أو الأشكال البيضاوية
- fillPolygon ، drawPolygon : لرسم المضلعات
- كل دوال الرسم تتطلب إحداثيات x و y ، و لكن معظمها تحتاج إلى العرض و الارتفاع.

ملاحظة آرسيزية:

- الدالة (Graphics و paint) تُستدعى تلقائياً من قبل النظام عندما يحتاج النظام لإعادة رسم محتويات المكون، مثل:
- عند تغيير حجم النافذة يتم استدعاء هذه الدالة داخل Frame أو Panel أو أي مكون آخر، ويتم تمرير كائن Graphics لها للرسم على المكون، وعند تنفيذها في AWT ، يتم أولاً استدعاء super.paint(g) وهذه الدالة تقوم بإعادة رسم المكونات الأساسية (مثل الخلفية)، وبعد استدعاء super.paint(g) يمكننا استخدام كائن Graphics لرسم الأشكال مثل المستطيلات وغيرها.

ملاحظة:

- في الحزمة AWT، لا توجد دالة مخصصة لرسم دائرة بشكل مباشر، لذلك يمكننا رسم دائرة باستخدام دالة drawOval، حيث أن الدائرة هي حالة خاصة من الشكل البيضاوي (oval) عندما يكون العرض (width) والارتفاع (height) متساويين (المربع أيضاً لا يوجد دالة لرسمه لذلك نرسم مستطيل وتكون أبعده متساوية) مثال:

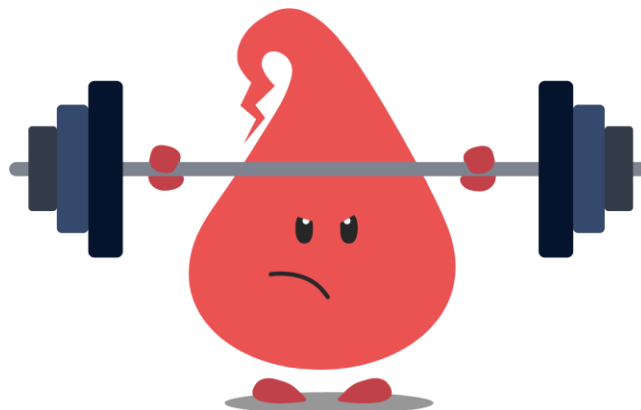
```
g.drawOval(150, 165, 100, 100);
```

Graphics methods that draw lines, rectangles and ovals:

■ تنويه: بالإضافة للجدول التالي يُفضل قراءة الجدول باللغة الإنكليزية من محاضرة الدكتور (الاسلايد 23 و24).

Method	Description
<code>public void drawLine (int x1, int y1, int x2, int y2)</code>	ترسم خطاً مستقيماً من النقطة (x1, y1) إلى النقطة (x2, y2) x1 و y1 : إحداثيات بداية الخط x2 و y2 : إحداثيات نهاية الخط
<code>public void drawRect(int x, int y, int width, int height)</code>	ترسم مستطيلاً فارغاً (الحدود فقط) عند الإحداثيات (x, y) بحجم معين. x,y : إحداثيات الزاوية العلوية اليسرى للمستطيل Width: عرض المستطيل Height: ارتفاع المستطيل
<code>public void fillRect(int x, int y, int width, int height)</code>	ترسم مستطيلاً مملوءاً عند الإحداثيات (x, y) بحجم معين x,y : إحداثيات الزاوية العلوية اليسرى للمستطيل Width: عرض المستطيل Height: ارتفاع المستطيل
<code>public void clearRect (int x, int y, int width, int height)</code>	إزالة محتوى منطقة مستطيل معين عن طريق تعبئتها باللون الافتراضي للخلفية. x,y : إحداثيات الزاوية العلوية اليسرى للمستطيل Width : عرض المستطيل Height : ارتفاع المستطيل
<code>public void drawRoundRect (int x, int y, int width, int height, int arcWidth, int arcHeight)</code>	ترسم مستطيلاً فارغاً بزوايا دائرية (الحدود فقط). x,y : إحداثيات الزاوية العلوية اليسرى Width : عرض المستطيل Height : ارتفاع المستطيل arcWidth : عرض القوس الذي يُحدد درجة انحناء الزوايا. arcHeight : ارتفاع القوس
<code>public void fillRoundRect (int x, int y, int width, int height, int arcWidth, int arcHeight)</code>	ترسم مستطيلاً مملوءاً بزوايا دائرية. x,y : إحداثيات الزاوية العلوية اليسرى Width : عرض المستطيل Height : ارتفاع المستطيل arcWidth : عرض القوس الذي يُحدد درجة انحناء الزوايا. arcHeight : ارتفاع القوس.

public void draw3DRect(int x, int y, int width, int height, boolean b)	ترسم مستطيلاً ثلاثي الأبعاد باستخدام الحدود فقط x,y : إحداثيات الزاوية العلوية اليسرى Width : عرض المستطيل Height : ارتفاع المستطيل b : إذا كانت true يظهر المستطيل وكأنه بارز للخارج، وإذا كانت false يبدو غائراً.
public void fill3DRect(int x, int y, int width, int height, boolean b)	ترسم مستطيلاً ثلاثي الأبعاد مملوءاً. x,y : إحداثيات الزاوية العلوية اليسرى Width : عرض المستطيل Height : ارتفاع المستطيل b : إذا كانت true يظهر المستطيل وكأنه بارز للخارج، وإذا كانت false يبدو غائراً.
public void drawOval(int x, int y, int width, int height)	ترسم شكلاً بيضوياً أو شكل قطع ناقص (الحدود فقط) x,y : إحداثيات الزاوية العلوية اليسرى للمستطيل الذي يحتوي على الشكل البيضوي Width : عرض القطع الناقص Height : ارتفاع القطع الناقص
public void fillOval (int x, int y, int width, int height)	ترسم شكلاً بيضوياً أو قطعاً ناقصاً مملوءاً x,y : إحداثيات الزاوية العلوية اليسرى للمستطيل الذي يحتوي على الشكل البيضوي Width : عرض القطع الناقص Height : ارتفاع القطع الناقص



■ مثال عن رسم المستطيلات والدوائر باستخدام الـ AWT:

```

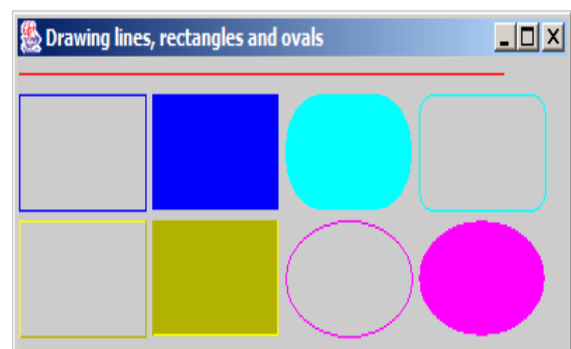
1 // يهدف البرنامج إلى رسم الخطوط، المستطيلات،
  والدوائر
2 import java.awt.*;
3 import javax.swing.*;
4 public class main extends JFrame {
5     // set window's title bar String and
  dimensions
6     public main() {
7         // كونسرتكتور الأب JFrame لتعيين عنوان النافذة
  إلى
  "Drawing lines, rectangles and ovals"
8         super ("Drawing lines, rectangles and
  ovals");
9         // تحديد أبعاد النافذة بـ 400 عرضاً و165 ارتفاعاً
10        setSize(400, 165);
11        // جعل النافذة مرئية
12        setVisible(true); }
13    public void paint(Graphics g) {
14        // تعريف الدالة paint(Graphics g) التي تُستخدم
  لرسم الأشكال
15        super.paint(g);
16        // ضبط اللون إلى الأحمر باستخدام
17        g.setColor(Color.RED);
18        // رسم خطاً يبدأ من النقطة (5، 30) وينتهي
  عند النقطة (350، 30)
19        g.drawLine(5, 30, 350, 30);
20        // ضبط اللون إلى الأزرق
21        g.setColor(Color.BLUE);
22        // رسم مستطيل يبدأ من النقطة (5، 40) بعرض
  55 وارتفاع 90
23        g.drawRect(5, 40, 90, 55);
24        // رسم مستطيل ممتلئ (Filled Rectangle)
  يبدأ من النقطة (100، 40) بنفس الأبعاد
25        g.fillRect(100, 40, 90, 55);
26        // ضبط اللون إلى السماوي (Cyan)
27        g.setColor(Color.CYAN);

```

```

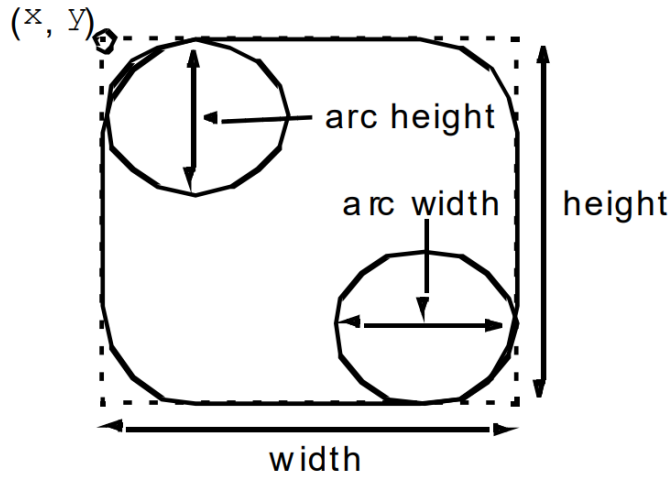
28        // رسم مستطيل مستدير الزوايا ممتلئ يبدأ
  من النقطة (195، 40) بعرض 90 وارتفاع 55، مع تحديد
  نصف قطر الزوايا بـ 50
29        g.fillRoundRect(195, 40, 90, 55, 50, 50);
30        // رسم مستطيل مستدير الزوايا غير ممتلئ يبدأ من
  النقطة (290، 40) بنفس الأبعاد، مع نصف قطر زوايا 20
31        g.drawRoundRect(290, 40, 90, 55, 20, 20);
32        // ضبط اللون إلى الأصفر
33        g.setColor(Color.YELLOW);
34        // رسم مستطيل ثلاثي الأبعاد غير ممتلئ
  (3D Rectangle) يبدأ من النقطة (5، 100)
35        // لجعل التأثير ثلاثي الأبعاد يبدو بارزاً: true
36        g.draw3DRect(5, 100, 90, 55, true);
37        // رسم مستطيلاً ثلاثي الأبعاد ممتلئاً يبدأ من
  النقطة (100، 100) بنفس الأبعاد
38        // لجعل التأثير ثلاثي الأبعاد يبدو غائراً: false
39        g.fill3DRect(100, 100, 90, 55, false);
40        // ضبط اللون إلى الأرجواني (Magenta)
41        g.setColor(Color.MAGENTA);
42        // رسم دائرة غير ممتلئة (Oval) داخل مستطيل
  يبدأ من النقطة (195، 100) بعرض 90 وارتفاع 55
43        g.drawOval(195, 100, 90, 55);
44        // رسم دائرة ممتلئة بنفس الأبعاد
45        g.fillOval(290, 100, 90, 55);}
46    public static void main(String args[])
47    {
48        main application = new main();
49        application.setDefaultCloseOperation
  (JFrame.EXIT_ON_CLOSE); }

```



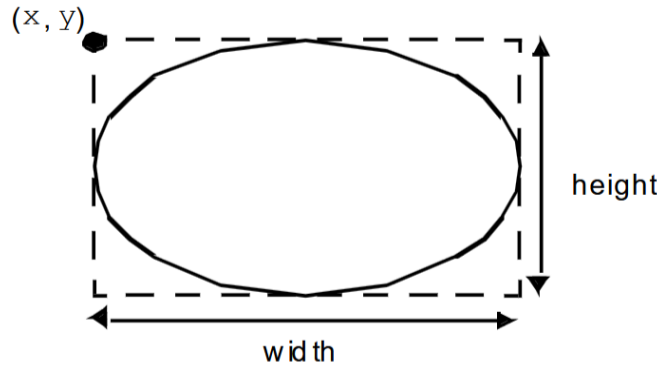
Arc width and arc height for rounded rectangles

الشكل التالي يوضح عرض وطول القوس الذي نستخدمه لتحديد زاوية انحناء المستطيل ذي الزوايا الدائرية:



Oval bounded by a rectangle

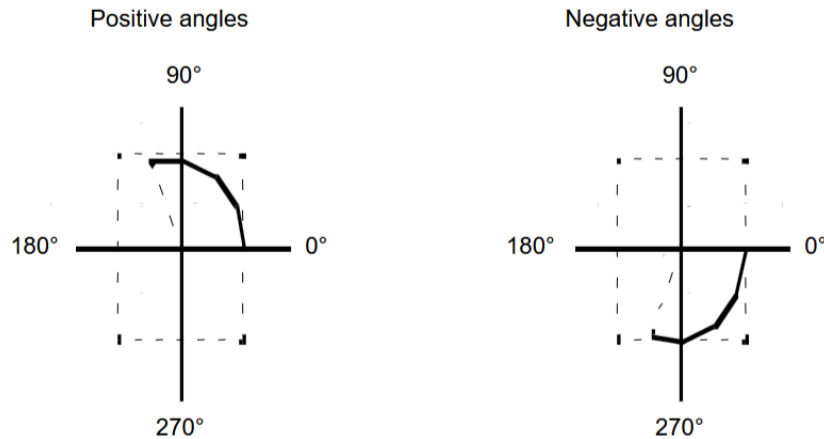
- القطع الناقص منحنٍ مغلق يمس مستطيلاً تخيلياً في جميع أضلاعه، لذا يمكن رسم دائرة أو قوس عن طريق اعتباره داخل مستطيل.
- الرسم يتم باستخدام دالة `drawOval` أو `fillOval` من كلاس `Graphics` ، هذه الدوال تقوم برسم دائرة أو قوس داخل مستطيل مع تحديد أبعاد المستطيل.



Drawing Arcs :

- القوس هو جزء (portion) من الشكل البيضاوي المحدد داخل مستطيل تخيلي، وهذا المستطيل يحدد موقع وأبعاد الشكل البيضاوي.
- زاوية البداية (starting angle) تُحدد من أين يبدأ القوس بالنسبة للمحور الأفقي.
- زاوية القوس تُحدد مدى الامتداد، سواء كانت موجبة أو سالبة:
 - موجبة: القوس يدور عكس اتجاه عقارب الساعة (Counterclockwise sweep).
 - سالبة: القوس يدور مع اتجاه عقارب الساعة (Clockwise sweep).
- تُقاس الزوايا بالدرجات / تُقاس من المحور الأفقي (اليمين) للشكل البيضاوي.

Positive and negative arc angles:



Graphics methods for drawing arcs:

تنويه: بالإضافة للجدول التالي يُفضل قراءة الجدول باللغة الإنكليزية من محاضرة الدكتور (الاسلايد 31).

Method	Description
public void drawArc(int x, int y, int width, int height, int startAngle, int arcAngle)	x و y : إحداثيات الزاوية العلوية اليسرى للمستطيل التخيلي الذي يحتوي على الشكل البيضاوي. width و height : العرض والارتفاع للمستطيل التخيلي الذي يحتوي على الشكل البيضاوي. startAngle : زاوية البداية للقوس. arcAngle : زاوية القوس (عدد الدرجات التي يتم تغطيتها بواسطة القوس). ملاحظة: هذه الدالة لرسم قوس كخط محيط (غير مملوء)
public void fillArc (int x, int y, int width, int height, int startAngle, int arcAngle)	x و y : إحداثيات الزاوية العلوية اليسرى للمستطيل التخيلي الذي يحتوي على الشكل البيضاوي. width و height : العرض والارتفاع للمستطيل التخيلي الذي يحتوي على الشكل البيضاوي. startAngle : زاوية البداية للقوس. arcAngle : زاوية القوس (عدد الدرجات التي يتم تغطيتها بواسطة القوس). القوس). ملاحظة: هذه الدالة لرسم قوس مملوء بلون

هذا البرنامج يُظهر كيفية رسم الأقواس (Arcs) // (Arcs)
 باستخدام مكتبة الرسومات في Java. الأقواس يمكن
 أن تكون مفتوحة أو ممتلئة (Filled Arcs) بألوان
 مختلفة وزوايا مختلفة

```

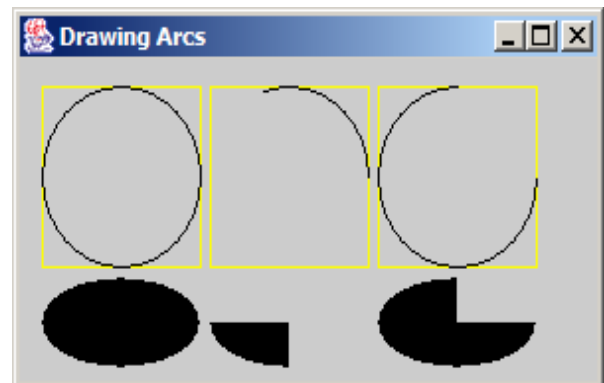
1 import java.awt.*;
2 import javax.swing.*;
3 public class DrawArc extends JFrame {
4     public DrawArc()
5     {
6         super("Drawing Arcs");
7         // تحديد أبعاد النافذة بـ 300 عرضًا و 170 ارتفاعًا
8         setSize(300, 170);
9         // جعل النافذة مرئية
10        setVisible(true); }
11        // تعريف الدالة paint(Graphics g) التي تُستخدم لرسم الأشكال
12        public void paint(Graphics g)
13        {
14            super.paint(g);
15            // ضبط اللون إلى الأصفر
16            g.setColor(Color.YELLOW);
17            // رسم مستطيل في النقطة (15, 35) بعرض
18            // وارتفاع 80، لتحديد منطقة رسم القوس.
19            g.drawRect(15, 35, 80, 80);
20            // تغيير اللون إلى الأسود
21            g.setColor(Color.BLACK);
22            // رسم قوس يغطي 360 درجة داخل
23            // المستطيل (يظهر كدائرة)
24            g.drawArc(15, 35, 80, 80, 0, 360);
25            g.setColor(Color.YELLOW);
26            // رسم مستطيل في النقطة (100, 35)
27            // بنفس الأبعاد
28            g.drawRect(100, 35, 80, 80);

```

```

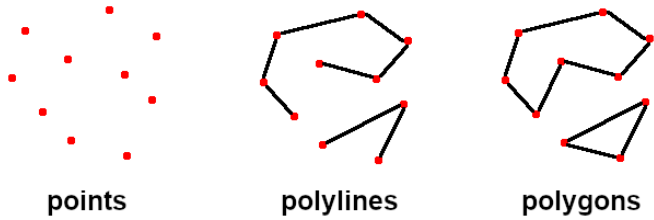
27        g.setColor(Color.BLACK);
28        // رسم قوس يغطي 110 درجات داخل المستطيل
29        g.drawArc(100, 35, 80, 80, 0, 110);
30        g.setColor(Color.YELLOW);
31        // رسم مستطيل في النقطة (185, 35) بنفس
32        // الأبعاد
33        g.drawRect(185, 35, 80, 80);
34        g.setColor(Color.BLACK);
35        // رسم قوس يغطي 270 درجة (يدور مع
36        // اتجاه عقارب الساعة)
37        g.drawArc(185, 35, 80, 80, 0, -270);
38        // رسم قوس ممتلئ داخل مستطيل يبدأ
39        // في النقطة (15, 120) بأبعاد 80x40 يغطي 360 درجة
40        // (يظهر كدائرة ممتلئة)
41        g.fillArc(15, 120, 80, 40, 0, 360);
42        // رسم قوس ممتلئ داخل مستطيل يبدأ
43        // في النقطة (100, 120) يغطي 90 درجة، يبدأ عند زاوية
44        // 270 درجة (من أسفل لأعلى)
45        g.fillArc(100, 120, 80, 40, 270, -90);
46        // رسم قوس ممتلئ داخل مستطيل يبدأ
47        // في النقطة (185, 120) يغطي 270 درجة
48        g.fillArc(185, 120, 80, 40, 0, -270); }
49        public static void main(String args[]){
50            DrawArc application = new DrawArc();
51            application.setDefaultCloseOperation
52            (JFrame.EXIT_ON_CLOSE); } }

```



Drawing Polygons and Polylines:

- Java توفر ميزة رسم الأشكال متعددة الأضلاع والخطوط المتعددة باستخدام Class Polygon مع استخدام كائنات Graphics
- (المضلعات) Polygons** : شكل مغلق يحتوي على سلسلة من الخطوط المستقيمة المتصلة، يمكن أن يكون مثلثاً، مربعاً، خماسياً، أو أي شكل هندسي له أكثر من 3 أضلاع (Multisided shapes).
- يتم تمثيل المضلعات باستخدام كائن Polygon و يتطلب هذا الكائن إحداثيات النقاط (X, Y) لتحديد رؤوس المضلع.
- (متعددات الخطوط) Polylines** : سلسلة من الخطوط المتصلة التي قد تكون مفتوحة / غير مغلقة (Series of connected points).



Graphics methods for drawing polygons and class Polygon methods

تنويه: بالإضافة للجدول التالي يُفضل قراءة الجدول باللغة الإنكليزية من محاضرة الدكتور (الاسلايد 36 و37).

Method	Description
public void drawPolygon(int xPoints[], int yPoints[], int points)	رسم محيط مضلع بناءً على مجموعة إحداثيات رؤوسه xPoints[] : مصفوفة تحتوي على الإحداثيات الأفقية (X) yPoints[] : مصفوفة تحتوي على الإحداثيات الشاقولية (Y) points : عدد النقاط التي تحدد المضلع
public void drawPolyline(int xPoints[], int yPoints[], int points)	رسم خطوط متصلة (غير مغلقة) بين النقاط المحددة
public void drawPolygon (Polygon p)	رسم مضلع باستخدام كائن Polygon
public void fillPolygon(int xPoints[], int yPoints[], int points)	رسم مضلع مملوء باللون الحالي
public void fillPolygon(Polygon p)	رسم مضلع مملوء باستخدام كائن Polygon
public Polygon()	إنشاء كائن مضلع
public Polygon (int xValues[], int yValues[], int numberOfPoints)	إنشاء مضلع بناءً على إحداثيات النقاط xValues[] : مصفوفة الإحداثيات الأفقية لرؤوس المضلع yValues[] : مصفوفة الإحداثيات الشاقولية لرؤوس المضلع numberOfPoints : عدد رؤوس المضلع
public void addPoint(int x, int y)	تضيف نقطة جديدة (رأس) إلى المضلع x : الإحداثي الأفقي للنقطة، y : الإحداثي الشاقولي للنقطة

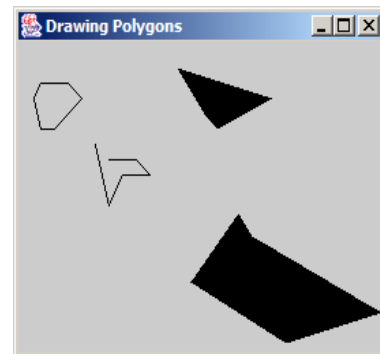
■ ملاحظة: عند استخدام إحدى دوال رسم المضلعات وفي حال كانت النقطة الأخيرة في مصفوفة الإحداثيات مختلفة عن النقطة الأولى يتم إغلاق المضلع بمستقيم واصل بين النقطتين.

■ مثال عن رسم المضلعات (Polygons) ومتعددي الخطوط (Polylines):

```

1  // البرنامج يرسم المضلعات
2  import java.awt.*;
3  import javax.swing.*;
4  public class DrawPolygons extends JFrame {
5      public DrawPolygons() {
6          super("Drawing Polygons");
7          // تحديد أبعاد النافذة بـ 275 عرضاً و 230 ارتفاعاً
8          setSize(275, 230);
9          // جعل النافذة مرئية
10         setVisible(true);
11         // تعريف الدالة paint(Graphics g) التي تُستخدم
12         // لرسم المضلعات وتم إنشاء reference من Graphics
13         public void paint(Graphics g) {
14             super.paint(g);
15             // إنشاء مصفوفة xValues تمثل الإحداثيات الأفقية لرؤوس المضلع
16             int xValues[] = { 20, 40, 50, 30, 20, 15 };
17             // إنشاء مصفوفة yValues تمثل الإحداثيات الشاقولية لرؤوس المضلع
18             int yValues[] = { 50, 50, 60, 80, 80, 60 };
19             // إنشاء كائن من النوع Polygon يمثل مضلعاً
20             // له 6 رؤوس
21             Polygon polygon1 = new Polygon(xValues, yValues, 6);
22             // رسم المضلع باستخدام drawPolygon
23             g.drawPolygon(polygon1);
24             // إنشاء مصفوفة xValues2 للإحداثيات الأفقية للنقاط
25             int xValues2[] = { 70, 90, 100, 80, 70, 65, 60 };
26             // إنشاء مصفوفة yValues2 للإحداثيات الشاقولية للنقاط
27             int yValues2[] = { 100, 100, 110, 110, 130, 110, 90 };
28             // رسم متعدد خطوط باستخدام drawPolyline
29             g.drawPolyline(xValues2, yValues2, 7);
30             // رسم مضلع مملوء باستخدام fillPolygon
31             // يملأ المساحة الداخلية للمضلع
32             g.fillPolygon(xValues3, yValues3, 4);
33             // إنشاء كائن Polygon فارغ
34             Polygon polygon2 = new Polygon();
35             // إضافة النقاط (الإحداثيات) إلى المضلع باستخدام addPoint
36             polygon2.addPoint(165, 135);
37             polygon2.addPoint(175, 150);
38             polygon2.addPoint(270, 200);
39             polygon2.addPoint(200, 220);
40             polygon2.addPoint(130, 180);
41             // رسم المضلع المملوء باستخدام fillPolygon
42             g.fillPolygon(polygon2);
43             public static void main(String args[]){
44                 DrawPolygons application = new DrawPolygons();
45                 application.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
46             }
47         }
48     }

```



The End